Serial Decode — Keithley DMM6500, DAQ6510, DMM7510, SMU2461

A UART decoder that runs **on** a Keithley bench instrument. It digitizes the line with the instrument's own digitizer, recovers the baud rate, frame format and idle polarity from the signal, and shows the bytes on the front panel as text or hex. No host, no logic analyser, one probe.

Ian Ameline · **version 1.20** · MIT licence (see LICENSE)

docs/MANUAL.md	using it: hooking up, the screen, the buttons, what it copes with, where it fails
docs/REFERENCE.md	every measured number, and the firmware limits behind them
docs/BENCH.md	the harness: the seeded sweep, the release gate stage by stage, reproducing a failure

Ships as `Serial_Decode.tspa`, installed from a USB key through the instrument's own **Manage Apps** screen. Written in TSP — Lua **5.0.2** embedded in the firmware, so the sources avoid `#`, `%`, `string.gmatch` and bitwise operators throughout.

What to know before trusting it

Tested on a DMM6500 and nothing else. Every measured number in these documents came off that one instrument. See Which instruments.

Flow control is verified electrically but never closed as a loop. One credit pulse per armed capture, measured on a scope, with nothing waiting for it on the other end. The width is not settable on this firmware, so a receiver needs an edge-triggered input rather than a polling loop. See **Triggering, in both directions** in REFERENCE.md.

Automatic rate detection has three known failures — **8 points in 860** on a full bench plan for the first two and about 3 in a million for the third — and all three are fixed by typing the rate in: a short pattern repeated over and over can measure at a small multiple of its true rate, a logic low near **–2 V** can misdetect, and above 115 200 baud a long run of one repeated byte can refuse. **None reaches an ordinary payload at a standard baud rate:** across 171 600 captures at standard rates, opened from 200 different points in the waveform, the fox, the 1 kB text payload, twelve random payloads and the walking-bit patterns are clean at every rate from 300 to 250 000 baud. All three are characterised under **Known failures of automatic rate detection** in the manual.

Nothing stops a long job early. A touch press cannot be delivered while a script runs, and the front-panel TRIGGER key does **not** deliver `trigger.EVENT_DISPLAY` during a panel-initiated run, so a recording runs to its stated size. That key does still *gate* an armed capture under `Options ▸ Trigger = Trigger key`.

One press is the whole interaction. A screenful is ~240 bytes; a recording holds 8 192 or 32 768 bytes to a file, decoded and filed without stepping; beyond that, flow control makes the window a chunk size and credits the device until it stops sending. Ceilings and arithmetic are in the manual.

Endurance, measured

15.5 hours with no computer attached, and zero events. The DMM6500 ran **10 671 cells over 8 complete laps** entirely on its own: it selected each waveform on the generator over the LAN, wrote its record to its own USB key, and had no host connected after the start. **0 captures instigated an instrument event** — which on this firmware means a modal box on the panel, and nothing suppresses it.

723 cells returned no result, **721 of them refusals the plan's own class allows**; of the other two, one was the generator failing to change waveform and one the app declining at 4–6.5 samples a bit, where declining is the better answer. **Confidently wrong bytes: none.**

Three 17-hour soaks before it, each 6 laps of the full 1 683-point matrix — 10 098 cycles on one power cycle — logged no instrument event, leaked no buffer or display object and left no error behind:

	duration	failures of 10 098	rate
first	17.14 h	318	3.15 %
second	17.17 h	313	3.10 %
third	17.18 h	301	2.98 %

Lap time did not rise in any of the three, which is the measurement that catches a leak: a leak wedges the instrument without failing a point. **Those failure counts are upper bounds** and they blend two unrelated failures; the per-cell records of these runs cannot be re-split. Measured separately, on the current build, over **1 066 400 offline decodes of the same plan: 95.9 % of failures are the reported rate being wrong, and 0.026 % of decodes get a byte wrong with the rate right.** Both, and what a plain payload at a standard rate does, are under **Rate detection and decode, counted separately** in REFERENCE.md.

Which instruments

The app needs a digitizer reachable as `dmm.digitize` or `smu.digitize`, the **touchscreen app API** (`display.create` and friends), and **Lua 5.0.2**. Every digitizer below runs at **1 MS/s**, and the **panel is pixel-identical across the whole TSP range**, so the app's screens carry over as they are.

DMM6500	**Tested**, firmware 1.7.17a — 15.5 hours driven by the instrument itself, three 17-hour host-driven soaks, and the namespace resolver verified here. 16-bit digitizer, "*maximum resolution 16 bits*", specifications, April 2018
DAQ6510	Should run unmodified, on the strongest grounds of any untested model: it **shares the UI board and the acquisition board** with the DMM6500, only the channel-board plugin differing. Untested
DMM7510	Should run unmodified: 18-bit digitizer, better acquisition boards. Untested
SMU2461	**Will install and try; may or may not work.** Dual 18-bit digitizers, reached as `smu.digitize` by a namespace the app resolves at load. That mechanism is verified on the DMM6500; no SMU has ever run it. Three unknowns below
2470 SMU	**No.** "*Digitized measurements are not a feature on the 2470*" — its reference manual, rev D, October 2024
2450 / 2460 SMU	Almost certainly not, by absence rather than denial: neither claims a digitizer, and the 2450 reference (rev D, May 2015) documents `smu.measure.*` with no digitize function

Porting is an acquisition question only. The panel is identical, so the display layer carries over untouched, and the app resolves `dmm.digitize` or `smu.digitize` once at load with no test on any capture path. Every rate figure in REFERENCE descends from the sample rate and the reading buffer's throughput, so the timing figures transfer; resolution is the one axis the app is indifferent to, thresholding each sample into a one or a zero. What to re-measure is what goes *through the acquisition board* — the tolerance envelope, the level and offset limits, the –2 V band. The DAQ6510 shares that board, so a deviation there is a real finding; the DMM7510's differs and those figures may move either way.

On an SMU2461 the app will install and try, and three things could stop it. *What the reading buffer holds* — a 2461 digitizes voltage and current **simultaneously** and the decoder trusts consecutive indices, so a buffer carrying current or source values alongside voltage reads as a waveform and surfaces as garbage bytes rather than an error. *The range* — the app pins **10 V**, chosen for the widest digitize bandwidth, which a 2461 may not offer. *The source output* — the app sets it off before every capture and **reads it back, refusing the capture if it cannot confirm it**; those constant names are unverified, and that refusal is the failure it is designed for.

The `.tspa` header decides where it installs, independently of whether the code would run: `$Product: DMM6500, DAQ6510, DMM7510, SMU2461`. Neither `DMM7510` nor `SMU2461` appears in the value set Keithley's app-header spec publishes, though Keithley ship TSP apps of their own declaring DMM7510 — an install failure is the symptom if a firmware validates the field strictly, and this field is the first thing to revert.

If you run this on anything but a DMM6500, please say so — open an issue. Every row above except the first is inference from API surfaces and vendor documents, and one report from real hardware outweighs all of it. Useful to include: `localnode.model` and `localnode.version`, whether **Manage Apps** offered the app at all, whether both screens built, and whether a capture decoded.

Before releasing it to anyone

Three gates, each about ten times the cost of the one before it. Run them in order.

```
python3 tools/release_sweep.py --offline   #  ~1 min, no instruments
python3 tools/bench_smoke.py               #  13 min, both instruments
python3 tools/soak.py --hours 17 --suites formats,plan --skip-vectors v95,v96
python3 tools/release_sweep.py             #  the whole thing, instruments included
```

The smoke gate covers the whole rate range in 13 minutes. Four waveforms — the fox and a random payload, each in 8N1 and 7E1 — paired crosswise, so one pair takes the 22 standard baud rates and the other the 21 drawn ones, and each rate family faces both formats and both content classes. Then all 45 button presses, seven logic swings from 5 V down to 0.25 V, and eight wrong-locked-rate cases. Measured: 86 cells in 9.7 min, 45 presses in 1.9, 7 levels in 0.7, 8 rate cases in 0.9.

No version is tagged without at least 8 hours of soak. A soak reports a **failure rate per test point**, where a sweep reports one pass or one failure. A lap is 1 683 cells and about 2.9 hours, so 8 hours is the least that separates "fails every lap" from "failed once", and 17 gives six laps.

Code changes do not get pushed without passing the smoke gate. On a pass, `bench_smoke.py` writes a receipt holding a hash of `tsp/` and `tools/`; a `pre-push` hook recomputes it and refuses if either tree has moved since. Documentation-only pushes are unaffected — neither tree is hashed — and when the bench is unavailable, `SMOKE_OVERRIDE="reason" git push` proceeds and prints the reason.

```
ln -sf ../../tools/hooks/pre-push .git/hooks/pre-push
```

Every lap draws its waveform order, its non-standard rates and the wait before each capture from the lap number, so `--iteration N` rebuilds any lap exactly without running the ones before it, and a failure replays offline in seconds.

docs/BENCH.md has the rest: every stage of the release gate and what it checks, the auditable record a full sweep leaves behind, what state the instruments have to be in before it will start, how to replay a failing cell offline, and the hazards worth knowing.

Layout

`tsp/`	the app. `serial_core` acquisition, `uart_decode` framing, `chunk_decode` resumable decode, `serial_ui` panel, `serial_app` orchestration
`tools/`	harnesses. `release_sweep.py` is the entry point; the rest are the authorities it calls
`docs/`	the manual, instrument references, panel mockups

`tsp/midi_decode.tsp` and `tsp/lin_decode.tsp` are complete and tested but **not shipped** in version 1 — the LIN checksum has never been checked against a real frame. Re-adding either is one line in `tools/package_tspa.py`; the app discovers what it has at runtime.

Bench

Two instruments over LAN: the DMM6500 under test and an SDG2000X-series generator producing the stimulus. **A scope is optional** — an SDS1000X-E is used here to confirm the stimulus is what it claims to be, and nothing in the gate needs it; it only ever reads. `tools/instruments.py` holds the addresses and the hazards: repeated large waveform uploads wedge the generator's LAN service, and calibration state is off limits on both.

Neither line's top model is needed. The bench asks the generator for 25 MSa/s of TrueArb, 20 Vpp and ~41 stored waveforms, and the scope, if used, for UART decode, two buses and 1 GSa/s; in both lines the model number is the *analog bandwidth*, and the fastest edge here is a 250 kBd bit. An **SDG2042X (40 MHz) and an SDS1104X-E (100 MHz) are sufficient**; the units here are an SDG2122X and an SDS1204X-E. `SDG_MAX_SRATE` is capped at 40 MSa/s so a plan cannot quietly outgrow the cheapest generator; see docs/BENCH.md for what to check on a unit that is not the one on this bench.